Fundamentos de Python

Estruturas de Dados



Universidade de Brasília
Departamento de Ciência da Computação

O que é Python?

- Criada por Guido van Rossum em 1991
- Classificada como de alto nível
- Versão atual 3.13
- Interpretada
- Vantagens:
  - Sintaxe simples e legível.
  - Grande comunidade e bibliotecas.



O que é Python?

O nome foi inspirado pelo programa de comédia britânico "Monty Python's Flying Circus".



O que é Python?

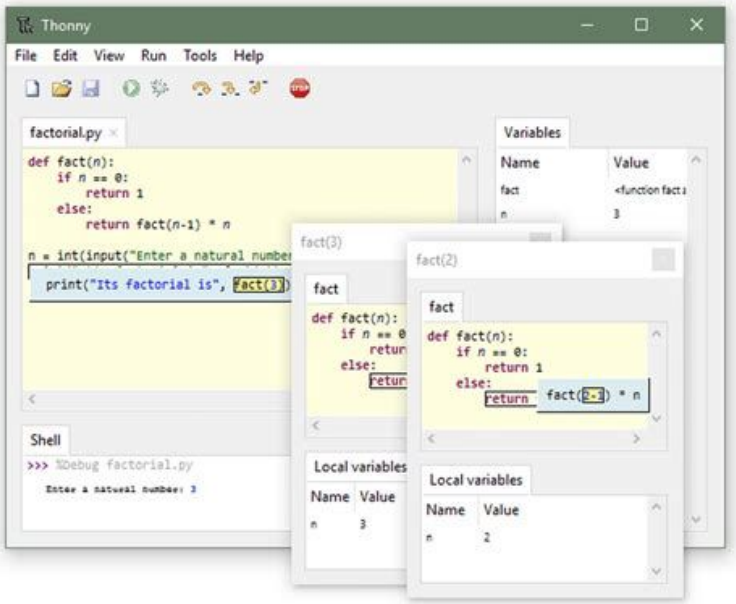
- Linguagem de programação de alto nível, fácil de aprender.
- Utilizada para
  - Automação
  - Desenvolvimento web
  - Análise de dados
  - Inteligência Artificial
  - entre outros.

Como rodar Python

- Instalação
  - www.python.org
  - Windows, Mac e Linux
  - Para Linux
    - `apt install python3.*`
    - Usar `pip` para gerenciar pacotes
  - Anaconda e Miniconda
    - https://www.anaconda.com/

Como rodar Python

- Interfaces de desenvolvimento
  - Thonny
  - PyCharm
  - VSCode
  - Jupyter Notebook
  - Repl.it



Sintaxe Básica (Comentários)

- Comentários de uma linha

```
# Isso é um comentário
```

- Comentários de múltiplas linhas

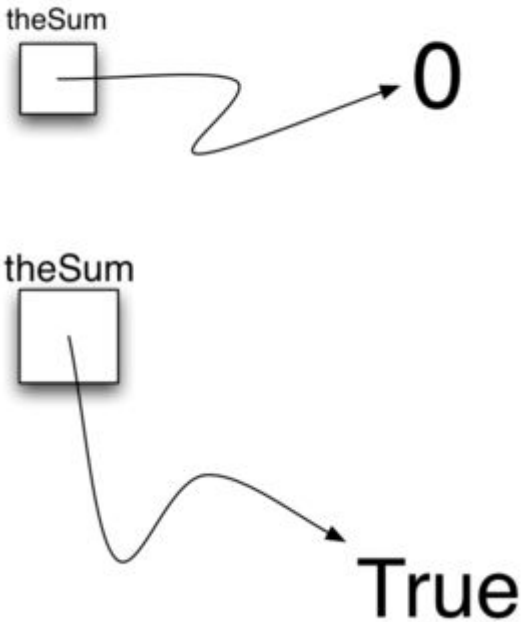
```
'''Isso pode ser
um comentário
de várias linhas'''
```

Sintaxe Básica (Identação)

- Identação
  - Python não usa `{}` ou `;` para blocos de código
  - Python usa espaços/tabs para definir blocos de código

Atribuição de variáveis

```
theSum = 0
theSum = True
```



Tipos Primitivos

- Inteiros (`int`)
- Ponto Flutuante (`float`)
- String (`str`)
- Booleanos (`bool`)

Tipos Primitivos

- Inteiros (`int`)
- Ponto Flutuante (`float`)
- String (`str`)
- Booleanos (`bool`)

Os tipos primitivos não têm um tamanho fixo em bytes, pois o gerenciamento de memória é dinâmico.

`int`

- Inteiros representam números sem parte decimal.
- Operações com inteiros: Soma, subtração, multiplicação, divisão inteira e exponenciação
- Exemplos:
  - `idade = 25`
  - `ano = 2024`
  - `população = 7800000000`

`float`

- Ponto flutuante representam números com parte decimal.
- Usado para cálculos que envolvem precisão decimal.
- Operações com ponto flutuante:
  - Soma, Subtração, Multiplicação retornam ponto flutuante
  - Divisão comum (/) sempre retorna um ponto flutuante, mesmo se os dois operandos forem inteiros.

`str`

- Strings
  - Sequências de caracteres
  - String são diferenciados dos identificadores usando aspas
  - Strings podem ser definidos com aspas simples (') ou duplas (").
  - Strings são imutáveis
  - `"aula"`

str

- Concatenação: juntar strings com +

```
nome_completo = "Luis" + " " + "Paulo"
```

- Repetição: multiplicar uma string para repetir o valor.

```
eco = "Olá! " * 3  # "Olá! Olá! Olá!"
```

- Acesso a caracteres: acessar elementos por índice.

```
letra = nome_completo[0]  # 'L'
```

`bool`

- Booleanos representam valores de verdade: `True` ou `False`.
- São muito usados em comparações e controle de fluxo
- Operações com booleanos:
  - `and`: retorna `True` se ambos os operandos forem `True`.
  - `or`: retorna `True` se pelo menos um dos operandos for `True`.
  - `not`: inverte o valor lógico.

Operadores

- Operadores Aritméticos

  `+, -, *, /, //, %, **`

- Operadores Relacionais

  `==, !=, >, <, >=, <=`

- Operadores Lógicos

  `and, or, not`

Controle de Fluxo

- Condicionais

  `If`, `elif` e `else`

- Laços de Repetição

  `for` e `while`

Condicionais

```
if condição:
    # Código
elif outra_condição:
    # Código
else:
    # Código
```

Condicionais

```
x = 10
if x > 5:
    print("Maior que 5")
elif x == 5:
    print("Igual a 5")
else:
    print("Menor que 5")
```

Laços de Repetição

```
for i in range(5):
    print(i)
```

Laços de Repetição

```
for i in range(5):
    print(i)
```



`range(start, stop, step)`:

- `start`: valor inicial (opcional, padrão 0).
- `stop`: valor final (não incluído).
- `step`: incremento (opcional, padrão 1).

Laços de Repetição

```
while condição:
    # código repetido
```

Funções Embutidas

- `print:` Exibe informações na tela.
- `input:` Recebe entrada do usuário.
- `type:` Retorna o tipo de um objeto.
- `len:` Retorna o tamanho de uma sequência

Entrada e Saída de Dados

```
nome = input("Digite seu nome: ")

print(f"Olá, {nome}!")
```

Bibliotecas em Python

- Ampliam as funcionalidades da linguagem.
- Existem bibliotecas padrão (inclusas no Python)
- Existem bibliotecas de terceiros (que devem ser instaladas).
  - Podem ser instaladas com o comando `pip`

Bibliotecas em Python

- **Importação completa:** Importa todas as funções e classes de um módulo.

```
import math

print(math.sqrt(16))
```

Bibliotecas em Python

- **Importação de funções específicas:** Importa apenas a função ou classe que você precisa.

```
from math import sqrt

print(sqrt(16))  # 4.0
```

Bibliotecas em Python

- **Importação com alias:** Útil para encurtar nomes de bibliotecas.

```
import numpy as np

array = np.array([1, 2, 3])
```

Obrigado!